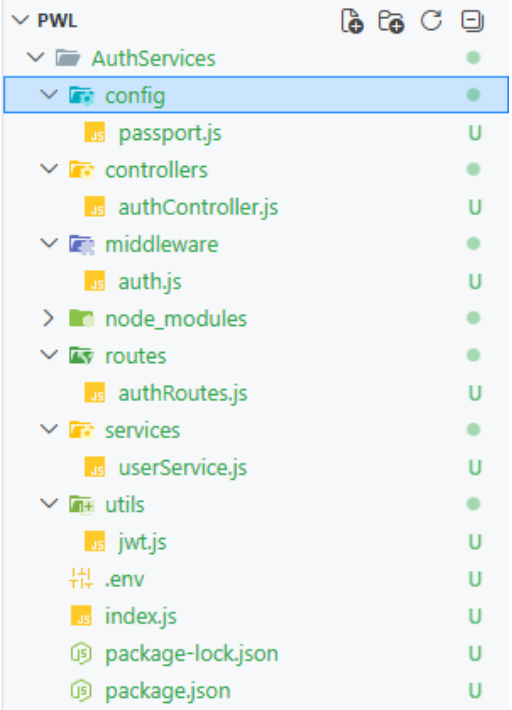
Nama: Joshua Taftfarel Petra Chrisensa

NPM: 232310002

Kelas: TI-23-KA



1. Env
   Berisi data-data penting yang menjadi konfigurasi dasar untuk membangun lingkungan project

AuthServices > .env

```
PORT=3003
NODE_ENV=development

GOOGLE_CLIENT_ID=813974346718-s42r2ufippiifqav962hnnk3lgkr1p9v.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=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
GOOGLE_CALLBACK_URL=http://localhost:3003/auth/google/callback

JWT_SECRET=1bik_b3st
JWT_EXPIRE=7d

SESSION_SECRET=t1_1bik_b3st

API_GATEWAY_URL=http://localhost:8080
```

2. Config

   Konfigurasi aplikasi pihat ketiga untuk menangani otentifikasi. Function Serialize dan Deserialize berfungsi untuk menyimpan data user

```
const passport = require("passport");
const GoogleStrategy = require("passport-google-oauth20").Strategy;
const userService = require("../services/userService");

// Serialize user - simpan user ID di session

passport.serializeUser((user, done) => {
  done(null, user._id || user.id);
});

// Deserialize user - ambil user dari user-services
passport.deserializeUser(async (id, done) => {
  try {
    const user = await userService.getUserById(id);
    done(null, user);
  } catch (error) {
    console.error("Deserialize error:", error);
    done(error, null);
  }
});

passport.use(
  new GoogleStrategy(
    {
      clientID: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
      callbackURL: process.env.GOOGLE_CALLBACK_URL,
      scope: ["profile", "email"],
    },
    async (accessToken, refreshToken, profile, done) => {
      try {
        console.log("📥 Google Profile received:", {
          id: profile.id,
          email: profile.emails[0].value,
          name: profile.displayName,
        });
        // Create or update user in user-services
        const user = await userService.createOrUpdateUser(profile);
        // Update login info
        await userService.updateLoginInfo(user._id);
        console.log("✓ User authenticated:", user.email);
        return done(null, user);
      } catch (error) {
        console.error("✗ Google Strategy Error:", error);
        return done(error, null);
      }
    },
  ),
);

module.exports = passport;
```

3. Utils
   a. generateToken = untuk melakukan pembuatan token pada user
   b. RefreshToken = untuk membuat ulang token baru user ketika token lama sudah expired
   c. VerifyToken = untuk mengecek apakah token valid dan belum expired atau tidak.

AuthServices > utils > jwt.js > ...

```
const jwt = require("jsonwebtoken");
/** * Generate JWT access token */ const generateToken = (user) => {
  return jwt.sign(
    {
      id: user._id || user.id,
      email: user.email,
      name: user.name,
      type: "access",
    },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRE || "7d" },
  );
};
/** * Generate JWT refresh token */ const generateRefreshToken = (user) => {
  return jwt.sign(
    { id: user._id || user.id, email: user.email, type: "refresh" },
    process.env.JWT_SECRET,
    { expiresIn: "30d" },
  );
};
/** * Verify JWT token */ const verifyToken = (token) => {
  try {
    return jwt.verify(token, process.env.JWT_SECRET);
  } catch (error) {
    throw new Error("Invalid or expired token");
  }
};
module.exports = { generateToken, generateRefreshToken, verifyToken };
```

4. Service
   - Insialisasi staticUser dan fungsi generate ID user

AuthServices > services > userService.js > ...

```javascript
// Static users database (in-memory)
const staticUsers = new Map();

// Helper function to generate user ID
const generateUserId = () => {
  return `user_${Date.now()}_${Math.random().toString(36).substr(2, 9)}`;
};

```

   - Untuk mengecek apakah email sudah ada atau belum. Jika sudah akan data diperbarui, jika belum akan generate ID baru bagi user

```javascript
const createOrUpdateUser = async (googleProfile) => {
  try {
    const email = googleProfile.emails[0].value;

    // Check if user already exists
    let user = Array.from(staticUsers.values()).find((u) => u.email === email);

    if (user) {
      // Update existing user
      user.name = googleProfile.displayName;
      user.firstName = googleProfile.name?.givenName;
      user.lastName = googleProfile.name?.familyName;
      user.picture = googleProfile.photos?.[0]?.value;
      user.verified = googleProfile._json?.email_verified || false;
      user.lastLogin = new Date();
      user.loginCount = (user.loginCount || 0) + 1;
      user.updatedAt = new Date();

      console.log("✓ User updated in static storage:", email);
    } else {
      // Create new user
      const userId = generateUserId();
      user = {
        _id: userId,
        googleId: googleProfile.id,
        email: email,
        name: googleProfile.displayName,
        firstName: googleProfile.name?.givenName,
        lastName: googleProfile.name?.familyName,
        picture: googleProfile.photos?.[0]?.value,
        verified: googleProfile._json?.email_verified || false,
        status: "active",
        role: "user",
        lastLogin: new Date(),
        loginCount: 1,
        createdAt: new Date(),
        updatedAt: new Date(),
      };

      staticUsers.set(userId, user);
      console.log("✓ New user created in static storage:", email);
    }

    return user;
  } catch (error) {
    console.error("✗ Error creating/updating user:", error.message);
    throw new Error("Failed to create/update user");
  }
};
```

   - Untuk memanggil/mendapatkan/memunculkan user berdasarkan pada id user

```
/**
 * Get user by ID from static storage
 */
const getUserById = async (userId) => {
  try {
    const user = staticUsers.get(userId);

    if (!user) {
      return null;
    }

    return user;
  } catch (error) {
    console.error("X Error getting user by ID:", error.message);
    throw new Error("Failed to get user");
  }
};

```

- Untuk memanggil user untuk mendapatkan data spesifik berdasarkan email

```
/**
 * Get user by email from static storage
 */
const getUserByEmail = async (email) => {
  try {
    const user = Array.from(staticUsers.values()).find(
      (u) => u.email === email,
    );
    return user || null;
  } catch (error) {
    console.error("X Error getting user by email:", error.message);
    throw new Error("Failed to get user by email");
  }
};
```

- Untuk memperbarui informasi login user sesui dengan id user yang melakukan login

```
/**
 * Update user login info
 */
const updateLoginInfo = async (userId) => {
  try {
    const user = staticUsers.get(userId);

    if (user) {
      user.lastLogin = new Date();
      user.loginCount = (user.loginCount || 0) + 1;
      user.updatedAt = new Date();

      console.log("✓ Login info updated for user:", user.email);
    }
  } catch (error) {
    console.error("X Error updating login info:", error.message);
    // Don't throw error, just log it
  }
};
```

- Untuk mengecek apakah user valid atau tidak

```
/**
 * Verify user exists and is active
 */
const verifyUser = async (userId) => {
  try {
    const user = await getUserById(userId);

    if (!user) {
      return { valid: false, message: "User not found" };
    }

    if (user.status !== "active") {
      return { valid: false, message: "User account is not active" };
    }

    return { valid: true, user };
  } catch (error) {
    return { valid: false, message: "Failed to verify user" };
  }
};
```

- Untuk memanggil semua user dan melakukan exports function

```
/**
 * Get all users (for debugging)
 */
const getAllUsers = () => {
  return Array.from(staticUsers.values());
};

module.exports = {
  createOrUpdateUser,
  getUserById,
  getUserByEmail,
  updateLoginInfo,
  verifyUser,
  getAllUsers,
};
```

5. Middleware

   Untuk menjaga keamanan data-data/endpoint-endpoint, dengan melakukan validasi/verify token user sesuai role

AuthServices > middleware > auth.js > ...

```javascript
const { verifyToken } = require("../utils/jwt");
const userService = require("../services/userService");
/** * Protect routes - require valid JWT token */

exports.protect = async (req, res, next) => {
  try {
    let token;
    // Check for token in headers
    if (
      req.headers.authorization &&
      req.headers.authorization.startsWith("Bearer")
    ) {
      token = req.headers.authorization.split(" ")[1];
    }
    // Check if token exists
    if (!token) {
      return res
        .status(401)
        .json({
          success: false,
          message: "Not authorized to access this route. Please login.",
        });
    }
    try {
      // Verify token
      const decoded = verifyToken(token);
      // Get user from user-services
      const user = await userService.getUserById(decoded.id);
      if (!user) {
        return res
          .status(401)
          .json({ success: false, message: "User not found" });
      }
      // Check if user is active
      if (user.status !== "active") {
        return res
          .status(403)
          .json({
            success: false,
            message: `Account is ${user.status}. Please contact administrator.`,
          });
      }
      // Attach user to request
      req.user = user;
      next();
    } catch (error) {
      return res
        .status(401)
        .json({ success: false, message: "Invalid or expired token" });
    }
  } catch (error) {
    console.error("Auth middleware error:", error);
    res
      .status(500)
      .json({
        success: false,
        message: "Error in authentication",
        error: error.message,
      });
  }
};
```

6. Controllers
   a. Insialisasi userService dan func token dari utils

AuthServices > controllers > authController.js > ...

```
const {
  generateToken,
  generateRefreshToken,
  verifyToken,
} = require("../utils/jwt");
const userService = require("../services/userService");

```

   b. Mengembalikan respon sukses dari google, membuat & me-refresh token, serta redirect user ke frontend

```
/**
 * @desc    Handle Google OAuth callback success
 * @route   GET /auth/google/callback
 * @access  Public
 */
exports.googleCallback = async (req, res) => {
  try {
    if (!req.user) {
      return res.redirect(
        `${process.env.FRONTEND_URL}/login?error=authentication_failed`,
      );
    }

    // Generate tokens
    const token = generateToken(req.user);
    const refreshToken = generateRefreshToken(req.user);

    // Redirect ke frontend dengan token
    const redirectUrl = `${process.env.FRONTEND_URL}/auth/callback?token=${token}&refreshToken=${refreshToken}`;

    console.log("✓ Google authentication successful for:", req.user.email);
    res.redirect(redirectUrl);
  } catch (error) {
    console.error("Google callback error:", error);
    res.redirect(`${process.env.FRONTEND_URL}/login?error=server_error`);
  }
};

```

   c. Melakukan pengiriman respon bila terjadi kegagalan

```
/**
 * @desc    Handle authentication failure
 * @route   GET /auth/google/failure
 * @access  Public
 */
exports.authFailure = (req, res) => {
  res.status(401).json({
    success: false,
    message: "Authentication failed",
  });
};

```

   d. Mengembalikan data profil user yang login

```
48  /**
49   * @desc    Get current logged in user
50   * @route   GET /auth/me
51   * @access  Private
52   */
53  exports.getMe = async (req, res) => {
54    try {
55      res.status(200).json({
56        success: true,
57        data: {
58          id: req.user._id,
59          googleId: req.user.googleId,
60          email: req.user.email,
61          name: req.user.name,
62          firstName: req.user.firstName,
63          lastName: req.user.lastName,
64          picture: req.user.picture,
65          verified: req.user.verified,
66          status: req.user.status,
67          lastLogin: req.user.lastLogin,
68          loginCount: req.user.loginCount,
69          createdAt: req.user.createdAt,
70          updatedAt: req.user.updatedAt,
71        },
72      });
73    } catch (error) {
74      console.error("Get profile error:", error);
75      res.status(500).json({
76        success: false,
77        message: "Server error",
78      });
79    }
80  };
81
```

e. Melakukan verifikasi token user

```
82  /**
83   * @desc    Verify JWT token
84   * @route   POST /auth/verify
85   * @access  Public
86   */
87  exports.verifyToken = async (req, res) => {
88    try {
89      const { token } = req.body;
90
91      if (!token) {
92        return res.status(400).json({
93          success: false,
94          message: "Token is required",
95        });
96      }
97
98      // Verify JWT
99      const decoded = verifyToken(token);
100
101     // Verify user dari user-services
102     const verification = await userService.verifyUser(decoded.id);
103
104     if (!verification.valid) {
105       return res.status(401).json({
106         success: false,
107         message: verification.message,
108       });
109     }
110
111     res.status(200).json({
112       success: true,
113       message: "Token is valid",
114       data: {
115         id: verification.user._id,
116         email: verification.user.email,
117         name: verification.user.name,
118         picture: verification.user.picture,
119         status: verification.user.status,
120       },
121     });
122   } catch (error) {
123     console.error("Verify token error:", error);
124     res.status(401).json({
125       success: false,
126       message: "Invalid or expired token",
127     });
128   }
129 };
```

f. Melakukan pembaharuan token user

```
131  /**
132   * @desc    Refresh access token
133   * @route   POST /auth/refresh
134   * @access  Public
135   */
136  exports.refreshToken = async (req, res) => {
137    try {
138      const { refreshToken } = req.body;
139
140      if (!refreshToken) {
141        return res.status(400).json({
142          success: false,
143          message: "Refresh token is required",
144        });
145      }
146
147      // Verify refresh token
148      const decoded = verifyToken(refreshToken);
149
150      if (decoded.type !== "refresh") {
151        return res.status(401).json({
152          success: false,
153          message: "Invalid refresh token",
154        });
155      }
156
157      // Verify user dari user-services
158      const verification = await userService.verifyUser(decoded.id);
159
160      if (!verification.valid) {
161        return res.status(401).json({
162          success: false,
163          message: verification.message,
164        });
165      }
166
167      // Generate new tokens
168      const newToken = generateToken(verification.user);
169      const newRefreshToken = generateRefreshToken(verification.user);
170
171      res.status(200).json({
172        success: true,
173        message: "Token refreshed successfully",
174        data: {
175          token: newToken,
176          refreshToken: newRefreshToken,
177        },
178      });
179    } catch (error) {
180      console.error("Refresh token error:", error);
181      res.status(401).json({
182        success: false,
183        message: "Invalid or expired refresh token",
184      });
185    }
186  };
187
```

g. Mematikan sesion user

```
/**
 * @desc    Logout user
 * @route   POST /auth/logout
 * @access  Private
 */
exports.logout = (req, res) => {
  try {
    req.logout((err) => {
      if (err) {
        console.error("Logout error:", err);
        return res.status(500).json({
          success: false,
          message: "Error during logout",
        });
      }

      req.session.destroy((err) => {
        if (err) {
          console.error("Session destroy error:", err);
        }

        res.status(200).json({
          success: true,
          message: "Logged out successfully",
        });
      });
    });
  } catch (error) {
    console.error("Logout error:", error);
    res.status(500).json({
      success: false,
      message: "Server error",
    });
  }
};
```

h. Memanggil user berdasarkan id yang dikirim

```
/**
 * @desc    Get user by ID (for internal service communication)
 * @route   GET /auth/user/:id
 * @access  Private
 */
exports.getUserById = async (req, res) => {
  try {
    const user = await userService.getUserById(req.params.id);

    if (!user) {
      return res.status(404).json({
        success: false,
        message: "User not found",
      });
    }

    res.status(200).json({
      success: true,
      data: user,
    });
  } catch (error) {
    console.error("Get user by ID error:", error);
    res.status(500).json({
      success: false,
      message: "Server error",
    });
  }
};
```

i. Memanggil statistik otentifikasi

```
/**
 * @desc    Get authentication statistics
 * @route   GET /auth/stats
 * @access  Private
 */
exports.getStats = async (req, res) => {
  try {
    // Karena kita tidak punya database sendiri,
    // kita bisa forward request ke user-services
    const response = await require("axios").get(
      `${process.env.USER_SERVICE_URL}/api/users/stats`,
      {
        headers: {
          Authorization: req.headers.authorization,
        },
      },
    );

    res.status(200).json({
      success: true,
      data: response.data.data,
    });
  } catch (error) {
    console.error("Get stats error:", error);
    res.status(500).json({
      success: false,
      message: "Server error",
    });
  }
};

```

j. Memanggil/menampilkan semua user yang ada dan valid

```
/**
 * @desc    Get all users (for debugging)
 * @route   GET /auth/users
 * @access  Private
 */
exports.getAllUsers = (req, res) => {
  try {
    const users = userService.getAllUsers();
    res.status(200).json({
      success: true,
      count: users.length,
      data: users,
    });
  } catch (error) {
    console.error("Get all users error:", error);
    res.status(500).json({
      success: false,
      message: "Server error",
    });
  }
};
```

k. Melakukan pengecekan kesehatan auth service

```
/**
 * @desc    Health check for auth service
 * @route   GET /auth/health
 * @access  Public
 */
exports.healthCheck = (req, res) => {
  res.status(200).json({
    success: true,
    message: "Auth service is healthy",
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    service: "auth-services",
    version: "1.0.0",
  });
};
```

7. Routes

   Penghubung antara url dengan controller

AuthServices > routes > authRoutes.js > ...

```javascript
const express = require("express");
const passport = require("passport");
const router = express.Router();
const authController = require("../controllers/authController");
const { protect } = require("../middleware/auth");

router.get(
  "/google",
  passport.authenticate("google", { scope: ["profile", "email"] }),
);

router.get(
  "/google/callback",
  passport.authenticate("google", {
    failureRedirect: "/auth/failure",
    session: true,
  }),
  authController.googleCallback,
);

router.get("/failure", authController.authFailure);

router.get("/me", protect, authController.getMe);
router.post("/verify", authController.verifyToken);
router.post("/refresh", authController.refreshToken);
router.post("/logout", protect, authController.logout);

router.get("/user/:id", protect, authController.getUserById);
router.get("/users", protect, authController.getAllUsers);

router.get("/health", authController.healthCheck);

module.exports = router;
```

8. Index

   Pintu gerbang untuk menjalankan program auth

   - Insialisasi variable & env, mengecek vairable env apakah ada yang hilang atau tidak
   - Mempersiapkan cors konfigurasi & memasukannya pada app dengan .use

AuthServices > index.js > ...

```
1  require("dotenv").config();
2  const express = require("express");
3  const session = require("express-session");
4  const cors = require("cors");
5  const passport = require("./config/passport");
6  const authRoutes = require("./routes/authRoutes");
7  const app = express();
8  // Validate required environment variables
9  const requiredEnvVars = [
10   "GOOGLE_CLIENT_ID",
11   "GOOGLE_CLIENT_SECRET",
12   "GOOGLE_CALLBACK_URL",
13   "JWT_SECRET",
14   "SESSION_SECRET",
15 ];
16 const missingEnvVars = requiredEnvVars.filter(
17   (varName) => !process.env[varName],
18 );
19 if (missingEnvVars.length > 0) {
20   console.error("❌ Missing required environment variables:");
21   missingEnvVars.forEach((varName) => {
22     console.error(` - ${varName}`);
23   });
24   process.exit(1);
25 }
26 // CORS Configuration
27 const corsOptions = {
28   origin: ["http://localhost:8080", process.env.FRONTEND_URL].filter(Boolean),
29   credentials: true,
30   optionsSuccessStatus: 200,
31   methods: ["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"],
32   allowedHeaders: ["Content-Type", "Authorization"],
33 };
34 app.use(cors(corsOptions));
35 // Body Parser Middleware
36 app.use(express.json());
37 app.use(express.urlencoded({ extended: true }));
```

   - Melakukan konfigurasi sesi, & inisialisasi passport
   - Memanggil dan menghubungkan routes & middleware

```
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
// Session Configuration
app.use(
  session({
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
      secure: process.env.NODE_ENV === "production",
      httpOnly: true,
      maxAge: 24 * 60 * 60 * 1000, // 24 hours
    },
  }),
);
// Initialize Passport
app.use(passport.initialize());
app.use(passport.session());
// Request Logger Middleware
app.use((req, res, next) => {
  console.log(`${new Date().toISOString()} - ${req.method} ${req.path}`);
  next();
});
// Routes
app.use("/auth", authRoutes);
// Root endpoint
app.get("/", (req, res) => {
  res.json({
    success: true,
    message: "Auth Service API",
    version: "1.0.0",
    timestamp: new Date().toISOString(),
    config: {
      googleClientId: process.env.GOOGLE_CLIENT_ID
        ? "✓ Configured"
        : "✗ Missing",
      googleClientSecret: process.env.GOOGLE_CLIENT_SECRET
        ? "✓ Configured"
        : "✗ Missing",
      callbackUrl: process.env.GOOGLE_CALLBACK_URL,
      jwtSecret: process.env.JWT_SECRET ? "✓ Configured" : "✗ Missing",
    },
    endpoints: {
      health: "/auth/health",
      test: "/auth/test",
      google: "/auth/google",
      callback: "/auth/google/callback",
      me: "/auth/me",
      verify: "/auth/verify",
      refresh: "/auth/refresh",
      logout: "/auth/logout",
      stats: "/auth/stats",
    },
  });
});
```

- 404 & error handler, serta penampil server ketika dijalankan

```
// 404 Handler
app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: "Route not found",
    path: req.path,
  });
});
// Error Handler
app.use((err, req, res, next) => {
  console.error("Error:", err);
  res.status(err.status || 500).json({
    success: false,
    message: err.message || "Internal server error",
    error: process.env.NODE_ENV === "development" ? err.stack : undefined,
  });
});
// Start Server
const PORT = process.env.PORT || 3003;
app.listen(PORT, () => {
  console.log("=".repeat(50));
  console.log("🚀 Auth Service Started");
  console.log("=".repeat(50));
  console.log(`📍 Server running on port: ${PORT}`);
  console.log(`🌍 Environment: ${process.env.NODE_ENV || "development"}`);
  console.log(`🔗 API URL: http://localhost:${PORT}`);
  console.log(`🔐 Google OAuth: http://localhost:${PORT}/auth/google`);
  console.log(`💚 Health Check: http://localhost:${PORT}/auth/health`);
  console.log("=".repeat(50));
  console.log("📋 Configuration Status:");
  console.log(` Google Client ID: ${process.env.GOOGLE_CLIENT_ID ? "✓" : "✗"}`);
  console.log(
    ` Google Client Secret: ${process.env.GOOGLE_CLIENT_SECRET ? "✓" : "✗"}`,
  );
  console.log(` Callback URL: ${process.env.GOOGLE_CALLBACK_URL}`);
  console.log("=".repeat(50));
});
// Handle unhandled promise rejections
process.on("unhandledRejection", (err) => {
  console.error("Unhandled Promise Rejection:", err);
  process.exit(1);
});
// Handle uncaught exceptions
process.on("uncaughtException", (err) => {
  console.error("Uncaught Exception:", err);
  process.exit(1);
});
module.exports = app;
```

9. Output
   a. API Url

Pretty-print ☑


```
{
  "success": true,
  "message": "Auth Service API",
  "version": "1.0.0",
  "timestamp": "2026-04-30T16:05:29.805Z",
  "config": {
    "googleClientId": "✓ Configured",
    "googleClientSecret": "✓ Configured",
    "callbackUrl": "http://localhost:3003/auth/google/callback",
    "jwtSecret": "✓ Configured"
  },
  "endpoints": {
    "health": "/auth/health",
    "test": "/auth/test",
    "google": "/auth/google",
    "callback": "/auth/google/callback",
    "me": "/auth/me",
    "verify": "/auth/verify",
    "refresh": "/auth/refresh",
    "logout": "/auth/logout",
    "stats": "/auth/stats"
  }
}
```


   b. Health Check

Pretty-print ☑


```
{
  "success": true,
  "message": "Auth service is healthy",
  "timestamp": "2026-04-30T16:05:42.000Z",
  "uptime": 25.6388022,
  "service": "auth-services",
  "version": "1.0.0"
}
```


   c. localhost:3003/auth/google

```
2026-04-30T16:06:01.565Z - GET /auth/google/callback
Google Profile received: {
  id: '100456018910183120421',
  email: 'joshuachrisensa@gmail.com',
  name: 'Joshua Chrisensa'
}
✓ New user created in static storage: joshuachrisensa@gmail.com
✓ Login info updated for user: joshuachrisensa@gmail.com
✓ User authenticated: joshuachrisensa@gmail.com
✓ Google authentication successful for: joshuachrisensa@gmail.com
2026-04-30T16:06:01.810Z - GET /auth/google/undefined/auth/callback
```